

Декодер DEFLATE (RFC 1951)

Головин Иван, Б05-304

1. Постановка задачи

1.1. Неформальная постановка

Надо написать программу, декодирующую данные, сжатые алгоритмом DEFLATE. Реализация выполняется на языке Haskell без внешних библиотек.

1.2. Формальная постановка

Пусть входные данные задаются строкой в кодировке Base64, которая передаётся единственным аргументом командной строки. Программа должна выводить декодированные данные в стандартный вывод.

1.2.1. Входная строка Base64

Алфавит Base64: $\Sigma_{64} = \{A \dots Z, a \dots z, 0 \dots 9, +, /\}$.

$$Base64String \rightarrow Group^* FinalGroup$$

$$Group \rightarrow c_1 c_2 c_3 c_4, \quad c_i \in \Sigma_{64}$$

$$FinalGroup \rightarrow c_1 c_2 c_3 c_4 \mid c_1 c_2 c_3 = \mid c_1 c_2 ==$$

Каждые четыре символа кодируют три байта (24 бита). Символ '=' является заполнителем и означает, что последняя группа кодирует меньше байт.

1.2.2. Поток байт DEFLATE

Согласно RFC 1951 поток байт DEFLATE представляет собой последовательность блоков:

$$DeflateStream \rightarrow Block^+$$

Каждый блок начинается заголовком из трёх бит:

Поле	Биты	Значение
bfinal	1	1 если это последний блок
btype	2	тип блока (00, 01, 10)

Типы блоков:

$$Block \rightarrow bfinal \ btype \begin{cases} StoredBlock & \text{если } btype = 00_2 \\ FixedBlock & \text{если } btype = 01_2 \\ DynamicBlock & \text{если } btype = 10_2 \end{cases}$$

Блок без сжатия (btype = 00₂):

$$StoredBlock \rightarrow \underbrace{\text{padding}}_{\text{выравнивание до байта}} \underbrace{LEN}_{2 \text{ байта}} \underbrace{NLEN}_{\text{дополнение}} \underbrace{byte^{LEN}}_{\text{данные}}$$

Блок с фиксированными кодами (btype = 01₂):

$$FixedBlock \rightarrow Symbol^* EOB$$

где таблицы кодов Хаффмана фиксированы стандартом (раздел 2.3.3).

Блок с динамическими кодами (btype = 10₂):

$$DynamicBlock \rightarrow \underbrace{HLIT}_{5 \text{ бит}} \underbrace{HDIST}_{5 \text{ бит}} \underbrace{HCLLEN}_{4 \text{ бита}} \underbrace{CodeLenCodes}_{HCLLEN+4 \text{ групп по 3 бита}} \underbrace{LitLenCodes}_{HLIT+257 \text{ значений}} \underbrace{DistCodes}_{HDIST+1 \text{ значений}} Symbol^* EOB$$

Символы (LZ77-пары):

$$Symbol \rightarrow \begin{cases} literal & \text{код } 0 \dots 255: \text{ байт данных} \\ EOB & \text{код } 256: \text{ конец блока} \\ LenCode DistCode & \text{код } 257 \dots 285: \text{ пара (длина, дистанция)} \end{cases}$$

1.2.3. Задача декодирования

Пусть $f_{b64} : \Sigma_{64}^* \rightarrow \{0, \dots, 255\}^*$ — функция декодирования Base64, $f_{def} : \{0, \dots, 255\}^* \rightarrow \{0, \dots, 255\}^*$ — функция декодирования DEFLATE.

Требуется реализовать функцию:

$$F(s) = f_{def}(f_{b64}(s)), \quad s \in \Sigma_{64}^*$$

и вывести $F(s)$, интерпретируя байты как символы UTF-8.

2. Используемые алгоритмы

2.1. Декодирование Base64

Base64 кодирует каждые 3 байта (24 бита) четырьмя символами из алфавита Σ_{64} , потому что каждый символ несёт 6 бит информации ($\log_2 64 = 6$).

Алгоритм.

1. Разбить строку на группы по 4 символа.
2. Для каждого символа c найти его номер $v(c) \in [0, 63]$.
3. Склеить 4 числа по 6 бит в 24-битное целое: $n = v(c_1) \cdot 2^{18} + v(c_2) \cdot 2^{12} + v(c_3) \cdot 2^6 + v(c_4)$.
4. Извлечь три байта: $b_1 = n \gg 16$, $b_2 = (n \gg 8) \& 0xFF$, $b_3 = n \& 0xFF$.
5. Если последняя группа оканчивается на '=': одним '=' - выдать 2 байта; двумя - выдать 1 байт.

2.2. Монадический парсер битового потока

Всё декодирование построено на типе:

type BitParser a = StateT BitState Maybe a

где BitState хранит список оставшихся байт и позицию следующего бита внутри текущего байта (0 = LSB). Использование монады StateT позволяет передавать состояние потока неявно через монадическую цепочку, а монада Maybe обеспечивает "отказ" при несоответствии входных данных.

Порядок битов в DEFLATE. Биты внутри байта читаются от младшего к старшему (LSB first). Коды Хаффмана при этом интерпретируются в обратном порядке: первый прочитанный бит является старшим битом кода.

2.3. Коды Хаффмана

2.3.1. Построение дерева по длинам кодов

RFC 1951 задаёт таблицы Хаффмана не явным перечислением кодов, а списком длин кодов для каждого символа. Дерево восстанавливается по следующему алгоритму:

1. Подсчитать $bl_count[l]$ — количество кодов длины l .
2. Вычислить начальный код для каждой длины:

$$next_code[1] = 0; \quad next_code[l + 1] = (next_code[l] + bl_count[l]) \cdot 2.$$

3. Присвоить каждому символу с длиной l очередной код $next_code[l]$, после чего увеличить $next_code[l]$ на 1.
4. Вставить тройки (символ, длина, код) в бинарное дерево, спускаясь по битам кода от старшего к младшему: бит 0 — в левого ребёнка, бит 1 — в правого.

2.3.2. Декодирование символа

Начиная с корня дерева, читать по одному биту и переходить в левого (бит=0) или правого (бит=1) ребёнка. Достигнув листа, вернуть хранящееся там значение.

2.3.3. Фиксированные таблицы

При $btype = 01_2$ таблица задана стандартом:

Символы	Длина кода	Примечание
0–143	8 бит	литералы (ASCII + расширенный)
144–255	9 бит	литералы выше 127
256–279	7 бит	ЕОВ + коды длин
280–287	8 бит	коды длин

Дистанции кодируются 5-битными кодами (30 значений, длина 5 для всех).

2.4. Алгоритм LZ77 (декодирование)

DEFLATE использует LZ77-сжатие: повторяющиеся фрагменты заменяются ссылкой (длина, дистанция). При декодировании надо:

1. Прочитать символ из таблицы литералов/длин.
2. Если символ < 256 : добавить байт в выходной буфер.
3. Если символ $= 256$: конец блока.
4. Если символ $\in [257, 285]$:
 - (a) Вычислить длину: $L = base_len[code - 257] + extra\ bits$.
 - (b) Прочитать код дистанции из дерева дистанций.
 - (c) Вычислить дистанцию: $D = base_dist[dist_code] + extra\ bits$.
 - (d) Скопировать L байт из позиции $(len(buf) - D)$ выходного буфера. Копирование может перекрываться (если $L > D$), что реализует RLE.

Таблицы кодов длин и дистанций в коде называются, соответственно, *lengthTable* и *distTable*

2.5. Декодирование динамических таблиц Хаффмана

При $btype = 10_2$ таблицы передаются в самом потоке, что позволяет адаптировать коды под конкретные данные.

Алгоритм.

1. Прочитать *HLIT* (5 бит), *HDIST* (5 бит), *HLEN* (4 бита). Количество кодов: литералы $= HLIT + 257$, дистанции $= HDIST + 1$, коды $= HLEN + 4$.
2. Прочитать $HLEN + 4$ трёхбитных значения в порядке: [16, 17, 18, 0, 8, 7, 9, 6, 10, 5, 11, 4, 12, 3, 13, 2, 14, 1, 15]. Это длины кодов для дерева.
3. Построить дерево Хаффмана.
4. С помощью дерева прочитать $(HLIT + 257) + (HDIST + 1)$ длин кодов:
 - Код 0...15: прямое значение длины.
 - Код 16: повторить предыдущую длину 3 + (2 бита) раз.
 - Код 17: 3 + (3 бита) нулей.
 - Код 18: 11 + (7 бит) нулей.
5. Построить деревья литералов и дистанций.

3. Архитектура программы

Программа организована как единственный модуль Main со следующими логическими блоками:

Блок	Ответственность
BitState / BitParser	Тип состояния и монада битового парсера
bit, bits, rawByte	Примитивы чтения битов и байт
decodeBase64	Декодирование входной строки Base64
HuffmanTree, buildFromList, assignCodes	Дерево Хаффмана и его построение
decodeSymbol	Чтение одного символа по дереву
fixedLitLenTree, fixedDistTree	Фиксированные таблицы RFC 1951
lengthTable, distTable	Таблицы длин и дистанций LZ77
decodeLZ	Основной цикл LZ77-декодирования
decodeDynTrees	Чтение динамических таблиц Хаффмана
decodeCodeLengths	RLE-раскодирование длин кодов
decodeStored	Раскодирование блока без сжатия
decodeBlocks	Цикл по блокам DEFLATE
decompressBase64	Точка входа в декодирование
main	Обработка аргументов командной строки

3.1. Конвейер обработки данных

$$\text{String (Base64)} \xrightarrow{f_{b64}} [\text{Word8}] \xrightarrow{\text{BitState}} \text{BitParser} \xrightarrow{\text{decodeBlocks}} [\text{Word8}] \xrightarrow{\text{toEnum}} \text{String}$$